

Innovative Management of Service Requests: From a Distributed Monolith to a Large-Scale Microservices Architecture, with RAG-Enhanced Recommendation System



Advisor:

Prof. Leonardo Mariani

Master's Thesis by:

Nicolas Guarini
Student ID 918670

Academic Year

2024/2025

Summary

This thesis describes the architectural redesign and restructuring of a legacy system for managing remotely executed Service Requests, developed during an internship at Elmec Informatica. The **primary objective of this work** was to transform an existing system, characterized by a distributed monolithic architecture with numerous structural and design limitations, into a modern, scalable, and modular microservices architecture named **ServiceRequestManager (SRM)**.

The project addressed significant technical challenges: a huge monolithic frontend with business logic hard-coded into the presentation layer; a heavy dependency on an Agent installed on customers' Domain Controllers and Exchange Servers, leading to security risks and operational bottlenecks; unversioned business logic stored in Stored Procedures and SSIS jobs; and critical data integrity issues in the legacy SQL Server database.

The objective was therefore to design and implement a solution that would overcome these limitations while ensuring business continuity, introducing significant technological innovations such as an intelligent recommendation sy-

stem based on Large Language Models and Retrieval-Augmented Generation (RAG) to assist users in navigating an ever-expanding catalog.

Analysis of the Existing System

The analysis of the legacy Service Request Portal (SRP) system revealed several architectural and technological criticalities accumulated over the years. The system was characterized by a fragmented architecture that compromised scalability, maintainability, and reliability.

The monolithic frontend represented an excessively complex component, where business logic was encoded directly into the presentation layer, making customizations nearly impossible without direct code intervention. The system relied heavily on an Agent installed on customers' Domain Controllers and Exchange Servers, which introduced security risks by requiring elevated permissions and created critical operational dependencies.

Data management presented equally significant issues: core logic was scattered across unversioned SQL Server Stored Procedures and SSIS jobs, making collaboration and error tracking difficult. The database schema suffered from significant redundancy and lack of referential integrity, resulting in data inconsistencies.

Architectural Redesign

To overcome the identified limitations, a new microservices-based architecture was designed, focused on modularity and scalability. The technological stack was modernized with the adoption of *Django* and *Django REST Framework* for the backend, and *PostgreSQL* replacing *MSSQL* to ensure strict data integrity and consistency.

The entire system was containerized via *Docker* and orchestrated using *Kubernetes*, enabling high availability, horizontal autoscaling, and zero-downtime deployments. Configuration and customization logic was moved from the frontend to the backend, reducing the user interface to approximately 1,000 lines of code and transforming it into a highly dynamic form builder driven by JSON definitions.

A central innovation of the SRM system is the implementation of a *three-level configuration hierarchy* (Fields $\rightarrow$ Actions $\rightarrow$ Customers), which enables granular customizations of service requests for specific clients and actions without code or logic duplication, ensuring a high degree of reusability. The dependency on legacy agents was eliminated through the integration of an agentless provisioning tool based on *Ansible*, significantly improving security and enabling versioning of all execution scripts in a centralized repository.

Implementation of the Modular Solution

The implementation placed particular emphasis on automating the development lifecycle through comprehensive CI/CD pipelines that automate code quality verification, security scanning, testing, building, and deployment across three distinct environments (devel, quality, production). The Git workflow requires feature branches to be integrated sequentially through these environments, ensuring progressive validation before production release.

The centralization of field management and configuration logic in the backend eliminated the monolithic frontend, enabling a highly reusable hierarchical configuration system. The integration of the new execution and retrieval flow for master data through a message queue system based on *RabbitMQ* was also essential for eliminating the dependency on the obsolete SRP Agent.

Coexistence and Gradual Migration Strategy

Recognizing the need to ensure business continuity, the thesis details a *phased coexistence and migration strategy*. The architecture implements a *dual-flow execution model* where individual actions can be routed through either the legacy SRP pipeline or the new SRM pipeline based on configuration.

This approach enabled a risk-managed transition through internal pilots and selected customer pilots, before full expansion and eventual decommissioning of the legacy infrastructure. Nightly synchronization jobs ensure that catalog data and execution history remain consistent across both systems during the migration period.

RAG-Based Recommendation System

The final and most innovative component of the work is the development of an *intelligent recommendation system* to assist users in navigating a catalog that has grown to over 400 actions. The system utilizes *Retrieval Augmented Generation (RAG)* combined with an on-premise Large Language Model (GPT-OSS-120B) and a *Chroma* vector store.

By processing user queries in natural language, the system identifies the most relevant actions and provides transparent reasoning for its suggestions, significantly reducing cognitive load on users and increasing trust in the automation. The architecture places particular emphasis on explainability through transparent reasoning, a critical requirement for enterprise adoption, distinguishing this implementation from consumer-facing recommendation systems.

Results Achieved

The project achieved significant results on multiple fronts. The architectural redesign resolved fundamental issues accumulated over the years: the mono-

lithic frontend component was reduced from 11,000 to approximately 1,000 lines of code through centralization of configuration logic in the backend. The three-level configuration hierarchy introduced unprecedented flexibility in catalog management, enabling customer-specific customizations without code duplication or catalog fragmentation.

The elimination of the SRP Agent dependency represented a major architectural breakthrough: integration with the Ansible-based provisioning system removed security risks associated with domain-admin credentials on customer infrastructure, eliminated polling-based communication inefficiencies, and enabled centralized logging.

The migration from unversioned Stored Procedures and SSIS jobs to version-controlled Python code within a comprehensive CI/CD pipeline transformed development practices. The adoption of PostgreSQL with properly enforced referential integrity constraints eliminated the data inconsistencies that plagued the legacy SQL Server database.

The RAG-enhanced recommendation system represents a significant innovation in enterprise service request management, enabling natural language-based action discovery that dramatically reduces cognitive load for end users. A comprehensive evaluation system was developed to quantitatively assess recommendation quality using a labeled dataset of 200 representative queries. Through iterative optimization driven by empirical evaluation, the system achieved 78.6% accuracy@1 (correct action as top recommendation) and 81.0% accuracy@3 (correct action within top-3 recommendations), demonstrating production readiness. The evaluation framework enabled data-driven refinement of RAG configuration parameters, resulting in a cumulative 16.3 percentage point improvement from the baseline implementation through optimizations including multilingual embedding models, stopword removal, chunk size calibration, and retrieval threshold tuning.

The gradual coexistence strategy successfully balanced innovation with operational stability, enabling incremental validation through internal pilots, customer pilots, and progressive expansion. The modular architecture facilitates future enhancements without requiring fundamental redesign, demonstrating the system's capacity to evolve alongside emerging technologies and changing business requirements.